

DineOS — Project Phases & Progress

Two tracks — Backend API and Flutter Frontend — each broken into phases. This is a status snapshot, not a plan requiring approval (the frontend phases were approved separately; see `Flutter_Frontend_Plan.pdf`).

1. The two tracks, at a glance

Backend (Django API) — <code>D:\DineOS-Backend</code>	
Core Flow (Phase 1)	DONE
Multi-Tenancy + Super Admin Retrofit	DONE
Phase 2 — Inventory, AI, Feedback, Reporting	NOT STARTED

Frontend (Flutter) — <code>C:\Users\skkar\DineOS-Frontend</code>	
Phase 0 — Foundation	DONE
Phase 1 — Customer Web + Server	IN PROGRESS
Phase 2 — Kitchen tablet + Cashier	NOT STARTED
Phase 3 — Admin / Manager	NOT STARTED
Phase 4 — Super Admin Web	NOT STARTED

2. Backend track — what each phase is

Phase	What it covers	Status
Core Flow	The sit-down restaurant loop end to end: QR scan → table session → multi-round ordering → kitchen lifecycle (New→Accepted→Preparing→Ready→Collected→Served) → billing → table freed. 6 roles (Admin, Manager, Server, Cashier, Kitchen, Customer). Real-time WebSocket push throughout.	DONE
Multi-Tenancy + Super Admin Retrofit	Turned the single-restaurant backend into one shared platform serving many client restaurants. Added a fully separate Super Admin auth realm to create tenants and flip four per-tenant paid add-on flags (Kitchen Display, Notifications, Billing, Realtime) live. Closed two real cross-tenant data leaks found during the retrofit.	DONE
Phase 2	Inventory/raw-ingredient tracking, real Groq/AI integration (currently mocked), customer feedback, cash-count/shift reports, full analytics reporting.	NOT STARTED

3. Frontend track — what each phase is

Phase	What it covers	Status
Phase 0	No user-facing screens yet — the foundation everything else builds on: Flutter SDK/toolchain, project scaffold (3 build targets), API client, auth (two separate realms mirroring the backend), design tokens, routing skeleton with role-based guards.	DONE
Phase 1	The core revenue loop, provable on a real table: Customer Web (QR scan → menu → cart → order → live status → request bill) and Server screens (table floor view → take order → ready-to-collect queue).	IN PROGRESS
Phase 2	Kitchen tablet (live order kanban board) + Cashier (bill preview, confirm payment) — completes the order lifecycle and closes the payment loop.	NOT STARTED
Phase 3	Admin/Manager screens: staff management, menu & category management, prepared-portsions, table history/reports.	NOT STARTED
Phase 4	Super Admin Web: tenant list, tenant detail, create tenant, flag toggles. No dependency on Phases 1–3 — could run in parallel.	NOT STARTED

4. What's actually been done so far

Backend — Core Flow (done, tested)

- Django + DRF + JWT auth (djangorestframework-simplejwt), Django Channels for WebSockets
- ~40 REST endpoints across auth, staff, tables/sessions, menu, prepared-dishes, orders, billing, notifications
- 3 WebSocket routes (staff, kitchen, table-session) for live push
- Full pytest + Postman/Newman test coverage, verified against a live running server each time

Backend — Multi-Tenancy + Super Admin (done, tested)

- `Restaurant` promoted to a real tenant model — slug, `is_active`, and 4 feature flags
- New `apps.platform` app: `PlatformAdmin` model, separate login/JWT claim, tenant CRUD + flag-toggle endpoints
- Tenant resolution middleware + explicit queryset scoping across every app; WebSocket groups and `notify_role()` namespaced per tenant
- Real Super Admin account created (`info@krypsos.tech`); demo/seed data confirmed wiped from the live DB
- Follow-up cleanup pass: dead settings wired in, PUT/DELETE test coverage added, a real bug found and fixed (category rename crashed with 500 instead of a clean 400), `/v1/auth/me/` extended to expose the caller's own tenant flags (a Flutter Phase 0 prerequisite)
- **Current state:** 44/44 pytest passing, 64/64 Postman requests passing, git tree clean at commit `c485562`

Frontend — Phase 0 (done, verified)

- Flutter 3.44.7 SDK installed and verified (`flutter doctor`)
- One codebase, 3 build targets: Staff App, Customer Web, Super Admin Web
- Dependencies: Riverpod (state), go_router (routing), dio (HTTP + JWT interceptor), web_socket_channel, flutter_secure_storage
- Two structurally separate auth realms (Staff vs Super Admin) mirroring the backend's own separation
- Design tokens + Material 3 theme; role-based routing guards
- 13 tests passing, `flutter analyze` clean, all 5 build targets verified via clean rebuild
- Real browser screenshots of all 6 Phase 0 screens captured and packaged (`DineOS-Frontend.zip` in Downloads)
- **Current state:** git tree clean at commit `e28d4bd`

Frontend — Phase 1 (in progress)

- Reviewed and confirmed the exact API/WebSocket contracts against the live backend code for every Phase 1 screen
- Noted a backend quirk to design around: broadcast WebSocket events don't carry an explicit "type" field, so the client dispatches by payload shape instead
- 3 of the shared domain models written so far: `MenuItem`, `RestaurantTable`, `OrderItem`

- **Not yet built:** remaining models (`TableSession` , `Order`), customer & server repositories/providers, the WebSocket client wrapper, all Customer and Server screens, routing updates, tests, and final verification

5. Documents produced so far

File	What it is
<code>docs/Super_Admin_Guide.pdf</code>	How the Super Admin realm works — identity, login, tenant control, feature licensing
<code>docs/Flutter_Frontend_Plan.pdf</code>	The approved frontend architecture/phasing plan this progress document tracks against
<code>DineOS-Frontend.zip</code> (Downloads)	Frontend source + real screenshots of every Phase 0 screen

DineOS — Project Phases & Progress · snapshot, not a plan requiring approval